

Curso Introducción a la Estadística con R

Clase 4: Estadística descriptiva

Factores y etiquetas

Etiquetas cuando importamos datos

- Cuando importamos datos que tienen etiquetas (por ejemplo de formatos como Stata o SPSS) debemos tener cuidado con cómo manejar estas etiquetas
- Por ejemplo, supongamos que queremos leer los datos de una encuesta con dos variables, guardada en formato Stata (.dta), con el paquete `haven`.

```
data <- haven::read_stata("data/ej_encuesta.dta")
head(data, 5)
```

```
## # A tibble: 5 × 2
##   P1          P14
##   <dbl+lbl> <dbl+lbl>
## 1 4 [Colonia] 1 [Muy mala]
## 2 18 [Tacuarembó] 2 [Mala]
## 3 15 [Salto] 5 [Muy buena]
## 4 1 [Artigas] 3 [Ni buena ni mala]
## 5 10 [Montevideo] 1 [Muy mala]
```

- Por defecto se leen como variables de tipo `double` (numérica) con etiquetas como atributos
- Si queremos quedarnos directamente con las etiquetas, podemos utilizar la función `as_factor`.

Factores

- Otro tipo de variables en R son los factores (factors), utilizados para representar data categórica. Estos suelen confundirse con las variables de caracteres pero tienen algunas diferencias.
- Normalmente los factores son utilizados para las variables de caracteres con un número de valores posibles fijo y cierto orden (opcional)
- A R le gusta transformar las variables de caracteres en factores al importarlas (si usamos R Base particularmente).
- El paquete `forcats` (dentro del Tidyverse) ayuda a manejar variables de caracteres y factores:
 - `fct_relevel()` cambia manualmente el orden de los niveles
 - `fct_reorder()` cambia el orden de los niveles de acuerdo a otra variable
 - `fct_infreq()` reordena un factor por la frecuencia de sus valores
 - `fct_lump()` collapse los valores menos frecuentes en otra categoría "other". Es muy útil para preparar datos para tablas y gráficos

Etiquetas cuando importamos datos

Si convertimos los datos a factor usando `haven::as_factor()` (no confundir con `as.factor()`) observamos las etiquetas de las variables.

```
data %>%  
  dplyr::mutate(P1 = haven::as_factor(P1),  
                P14 = haven::as_factor(P14)) %>%  
  head()
```

```
## # A tibble: 6 × 2  
##   P1      P14  
##   <fct>   <fct>  
## 1 Colonia Muy mala  
## 2 Tacuarembó Mala  
## 3 Salto    Muy buena  
## 4 Artigas  Ni buena ni mala  
## 5 Montevideo Muy mala  
## 6 Montevideo Buena
```

Factores

```
# Podemos chequear y coercionar factores
data_gapminder <- gapminder
is.factor(data_gapminder$continent) # Chequeo si es factor
```

```
## [1] TRUE
```

```
levels(data_gapminder$continent) # Chequeo los niveles
```

```
## [1] "Africa" "Americas" "Asia" "Europe" "Oceania"
```

```
# Transformo a caracter
data_gapminder$continent <- as.character(data_gapminder$continent)
class(data_gapminder$continent)
```

```
## [1] "character"
```

```
# De vuelta a factor
data_gapminder$continent <- as.factor(data_gapminder$continent)
class(data_gapminder$continent)
```

```
## [1] "factor"
```

Factores

```
# Para crear un factor usamos la función factor()
países_mercosur <- factor(c("Argentina", "Brasil", "Paraguay", "Uruguay"))
table(países_mercosur)
```

```
## países_mercosur
## Argentina      Brasil  Paraguay   Uruguay
##           1           1           1           1
```

```
# La función fct_relevel() nos permite reordenar los niveles del factor
países_mercosur <- fct_relevel(países_mercosur, "Uruguay")
table(países_mercosur)
```

```
## países_mercosur
##   Uruguay Argentina      Brasil  Paraguay
##         1           1           1           1
```

Dataframes

Dataframes: tibbles

- La mayoría de los análisis de datos convencionales contienen dataframes. Cuando usamos los paquetes del tidyverse, generalmente trabajamos con "tibbles", que es muy similar a un dataframe pero con pequeños cambios.
- Una de las principales diferencias es la forma en que se imprimen los datos.
- La mayoría de las funciones del Tidyverse devuelven un tibble.

```
data_gapminder <- (gapminder)
class(data_gapminder) # Ya es un tibble
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

```
data_gapminder <- as.data.frame(data_gapminder)
class(data_gapminder) # Ahora solamente dataframe
```

```
## [1] "data.frame"
```

Dataframes: imprimir dataframe

```
print(data_gapminder)
```

	country	continent	year	lifeExp	pop	gdpPercap
## 1	Afghanistan	Asia	1952	28.80100	8425333	779.4453
## 2	Afghanistan	Asia	1957	30.33200	9240934	820.8530
## 3	Afghanistan	Asia	1962	31.99700	10267083	853.1007
## 4	Afghanistan	Asia	1967	34.02000	11537966	836.1971
## 5	Afghanistan	Asia	1972	36.08800	13079460	739.9811
## 6	Afghanistan	Asia	1977	38.43800	14880372	786.1134
## 7	Afghanistan	Asia	1982	39.85400	12881816	978.0114
## 8	Afghanistan	Asia	1987	40.82200	13867957	852.3959
## 9	Afghanistan	Asia	1992	41.67400	16317921	649.3414
## 10	Afghanistan	Asia	1997	41.76300	22227415	635.3414
## 11	Afghanistan	Asia	2002	42.12900	25268405	726.7341
## 12	Afghanistan	Asia	2007	43.82800	31889923	974.5803
## 13	Albania	Europe	1952	55.23000	1282697	1601.0561
## 14	Albania	Europe	1957	59.28000	1476505	1942.2842
## 15	Albania	Europe	1962	64.82000	1728137	2312.8890
## 16	Albania	Europe	1967	66.22000	1984060	2760.1969
## 17	Albania	Europe	1972	67.69000	2263554	3313.4222
## 18	Albania	Europe	1977	68.93000	2509048	3533.0039
## 19	Albania	Europe	1982	70.42000	2780097	3630.8807
## 20	Albania	Europe	1987	72.00000	3075321	3738.9327
## 21	Albania	Europe	1992	71.58100	3326498	2497.4379
## 22	Albania	Europe	1997	72.95000	3428038	3193.0546
## 23	Albania	Europe	2002	75.65100	3508512	4604.2117
## 24	Albania	Europe	2007	76.42300	3600523	5937.0295
## 25	Algeria	Africa	1952	43.07700	9279525	2449.0082
## 26	Algeria	Africa	1957	45.68500	10270856	3013.9760
## 27	Algeria	Africa	1962	48.30300	11000948	2550.8169
## 28	Algeria	Africa	1967	51.40700	12760499	3246.9918
## 29	Algeria	Africa	1972	54.51800	14760787	4182.6638
## 30	Algeria	Africa	1977	58.01400	17152804	4910.4168
## 31	Algeria	Africa	1982	61.36800	20033753	5745.1602
## 32	Algeria	Africa	1987	65.79900	23254956	5681.3585
## 33	Algeria	Africa	1992	67.74400	26298373	5023.2166
## 34	Algeria	Africa	1997	69.15200	29072015	4797.2951
## 35	Algeria	Africa	2002	70.99400	31287142	5288.0404
## 36	Algeria	Africa	2007	72.30100	33333216	6223.3675
## 37	Angola	Africa	1952	30.01500	4232095	3520.6103
## 38	Angola	Africa	1957	31.99900	4561361	3827.9405
## 39	Angola	Africa	1962	34.00000	4826015	4269.2767
## 40	Angola	Africa	1967	35.98500	5247469	5522.7764
## 41	Angola	Africa	1972	37.92800	5894858	5473.2880
## 42	Angola	Africa	1977	39.48300	6162675	3008.6474
## 43	Angola	Africa	1982	39.94200	7016384	2756.9537
## 44	Angola	Africa	1987	39.90600	7874230	2430.2083

Dataframes: imprimir tibble

```
data_gapminder <- as_tibble(data_gapminder) # Pasamos nuevamente a tibble
class(data_gapminder)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

```
print(data_gapminder)
```

```
## # A tibble: 1,704 × 6
##   country      continent year lifeExp      pop gdpPercap
##   <fct>        <fct>    <int>   <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952    28.8  8425333    779.
## 2 Afghanistan Asia      1957    30.3  9240934    821.
## 3 Afghanistan Asia      1962    32.0 10267083    853.
## 4 Afghanistan Asia      1967    34.0 11537966    836.
## 5 Afghanistan Asia      1972    36.1 13079460    740.
## 6 Afghanistan Asia      1977    38.4 14880372    786.
## 7 Afghanistan Asia      1982    39.9 12881816    978.
## 8 Afghanistan Asia      1987    40.8 13867957    852.
## 9 Afghanistan Asia      1992    41.7 16317921    649.
## 10 Afghanistan Asia      1997    41.8 22227415    635.
## # i 1,694 more rows
```

Tidy dataset

- Hay muchas formas de estructurar un conjunto de datos. El enfoque tidy sugiere que cada variable sea una columna y cada observación sea una fila, por lo que cada valor tiene su propia celda:

country	year	cases	population
Afghanistan	1999	745	15557071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	17206362
Brazil	2000	80488	17404898
China	1999	212258	1272915272
China	2000	216766	128042583

variables

country	year	cases	population
Afghanistan	1999	745	15557071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	17206362
Brazil	2000	80488	17404898
China	1999	212258	1272915272
China	2000	216766	128042583

observations

country	year	cases	population
Afghanistan	1999	745	15557071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	17206362
Brazil	2000	80488	17404898
China	1999	212258	1272915272
China	2000	216766	128042583

values

Wickham & Grolemund (2018)

Nombres de variables

Muchas veces los usamos datos que no están documentados de manera uniforme o apropiada, por ejemplo, con nombres dispares y propensos a errores en las columnas.

Janitor es un paquete orientado al estilo Tidyverse (aunque no pertenece) que facilita algunas funciones para limpiar y explorar datos.

ejemplo

```
##      COLORES NombresCompletos edad_NUMERICA
## 1 Verde      María S.           32
## 2 Rojo       Juan F.           23
## 3 Azul       Pedro A.          24
```

```
library(janitor)
```

```
ejemplo_clean <- clean_names(ejemplo)
ejemplo_clean
```

```
##      colores nombres_completos edad_numerica
## 1 Verde      María S.           32
## 2 Rojo       Juan F.           23
## 3 Azul       Pedro A.          24
```

class: inverse, center, middle

Explorar Dataframes

Resumen de un dataframe

```
data_gapminder <- gapminder
```

```
# R tiene un visor para datos. Pueden dar click  
# en el dataframe en el ambiente o:  
view(data_gapminder)
```

```
dim(data_gapminder) # Número de filas y columnas
```

```
## [1] 1704    6
```

```
names(data_gapminder) # Nombre de variables
```

```
## [1] "country" "continent" "year" "lifeExp" "pop" "gdpPercap"
```

```
head(data_gapminder, 3) # Imprime primeras filas (3 en este caso)
```

```
## # A tibble: 3 × 6  
##   country    continent year lifeExp      pop gdpPercap  
##   <fct>      <fct>    <int>   <dbl>   <int>    <dbl>  
## 1 Afghanistan Asia      1952    28.8  8425333    779.  
## 2 Afghanistan Asia      1957    30.3  9240934    821.  
## 3 Afghanistan Asia      1962    32.0 10267083    853.
```

Resumen de un dataframe

Una de las funciones más útiles para resumir un dataframe es `glimpse()` del paquete `dplyr` o `tidyverse`. Es particularmente útil debido a que permite un vistazo al nombre, tipo y primeros valores de **todos** las variables de un dataframe.

```
# Resumen más completo:  
glimpse(gapminder)
```

```
## Rows: 1,704  
## Columns: 6  
## $ country   <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ...  
## $ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ...  
## $ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ...  
## $ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8...  
## $ pop       <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12...  
## $ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ...
```

```
]
```

Transformar datos

Transformar datos con dplyr

El paquete dplyr contiene funciones muy útiles para la transformación de dataframes (tibbles). Todas las funciones tienen en común que su primer argumento es un dataframe y que devuelven un dataframe. Algunas de las funciones que vamos a ver:

- `filter()`: filtrar observaciones en base a valores
- `select()`: filtrar variables
- `mutate()`: crear o recodificar variables
- `group_by()`: define grupos de valores utilizar las otras funciones
- `summarise()`: colapsa valores según alguna fórmula (sumar, número de casos, media, etc.)

Filtrar

Una de las tareas más comunes en el análisis de datos es filtrar observaciones en base a condiciones. Existen muchas maneras de filtrar datos en R, la función `filter()` de dplyr es una de las más sencillas de utilizar. El primer argumento es el dataframe y el segundo la condición por la que queremos filtrar.

```
# Tenemos datos de muchos años:  
table(gapminder$year)
```

```
##  
## 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 2002 2007  
##  142  142  142  142  142  142  142  142  142  142  142  142
```

```
# Filtramos para con los datos a partir de 2007  
gapminder_07 <- filter(gapminder, year == 2007)  
table(gapminder_07$year)
```

```
##  
## 2007  
##  142
```

Utilizando operadores lógicos podemos filtrar de formas más complejas:

```
# Todas los años menos 2007
gapminder_pre07 <- filter(gapminder, year != 2007)
table(gapminder_pre07$year)
```

```
##
## 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 2002
##  142  142  142  142  142  142  142  142  142  142  142
```

```
# Solo los siguientes años: 1952, 1992 y 2007
años_especificos <- c(1952, 1992, 2007)
gapminder_esp <- filter(gapminder, year %in% años_especificos)

table(gapminder_esp$year)
```

```
##
## 1952 1992 2007
##  142  142  142
```

Seleccionar variables

Con `select()` podemos seleccionar las variables (columnas) que queremos mantener en un dataframe. Podemos nombrarlas, seleccionar cuáles queremos eliminar y referirnos por su orden:

```
# Seleccionar un conjunto de variables (país, año, población)  
select(gapminder, country, year, pop)  
  
# Seleccionar todas las variables menos las especificadas  
select(gapminder, -continent)  
  
# Seleccionar un rango de variables según orden  
select(gapminder, country:lifeExp)  
select(gapminder, 1:3) # Orden numérico
```

Pipeline %>%

Cuando queremos realizar más de una operación a un dataframe podemos utilizar el pipeline. Como vimos, la mayoría de las funciones de dplyr que se aplican a un dataframe tienen como primer argumento el dataframe al que le queremos aplicar la función.

Con el pipeline especificamos el dataframe solamente una vez al principio, y luego todas las funciones que vamos utilizando no necesitan especificación. De esta forma nos enfocamos en la transformación y no en el objeto.

```
gapminder_07_america <- gapminder %>%  
  filter(year == 2007 & continent == "Americas") %>%  
  select(-continent)  
  
print(gapminder_07_america)
```

```
## # A tibble: 25 × 5  
##   country      year lifeExp      pop gdpPercap  
##   <fct>      <int>   <dbl>   <int>   <dbl>  
## 1 Argentina  2007    75.3  40301927  12779.  
## 2 Bolivia    2007    65.6   9119152   3822.  
## 3 Brazil     2007    72.4 190010647   9066.  
## 4 Canada     2007    80.7  33390141  36319.  
## 5 Chile      2007    78.6  16284741  13172.  
## 6 Colombia   2007    72.9  44227550   7007.  
## 7 Costa Rica 2007    78.8   4133884   9645.  
## 8 Cuba       2007    78.3  11416987   8948.  
## 9 Dominican Republic 2007    72.2   9319622   6025.  
## 10 Ecuador   2007    75.0  13755680   6873.  
## # i 15 more rows
```

Pipeline %>%

- Una de las ventajas del Tidyverse es la facilidad con la que se puede leer e interpretar el código. Un elemento fundamental para esto es el pipeline (%>%). Es muy útil para expresar una secuencia de muchas operaciones.
- Habíamos visto varias formas de realizar esto: sobrescribir el mismo objeto, con objetos intermedios o anidando funciones.
- El pipeline del paquete **magrittr** hace más fácil modificar operaciones puntuales dentro de conjunto de operaciones, hace que sea más fácil leer (evitando leer de adentro hacia afuera) entre otras ventajas.
- Es recomendable evitar usar el pipeline cuando queremos trabajar más de un objeto a la vez
- `x %>% f == f(x)`
- Se puede leer como un "y entonces"

Otras funciones de dplyr muy útiles

- `arrange()` ordenar los datos según una o más variables
- `rename()` cambiar el nombre de las variables de un dataframe
- `pull()` y `distinct()`: con `distinct()` es posible identificar los valores distintos de una variable y con `pull()` los podemos extraer como un vector
- `slice_min()` y `slice_max()`: filtrar n observaciones de mayor o menor valor según variable. En general, la familia de funciones `slice` permite filtrar observaciones en función de su posición.
- `count()` contar observaciones por grupo
- `relocate()` cambiar el orden de columnas

Crear y recodificar variables

Crear variables con mutate()

El paquete **dplyr** contiene la función `mutate()` para crear nuevas variables. `mutate()` crea variables al final del dataframe.

```
data_gapminder <- gapminder

# Variable de caracteres
data_gapminder <- mutate(data_gapminder, var1 = "Valor fijo")

# Variable numérica
data_gapminder <- mutate(data_gapminder, var2 = 7)
head(data_gapminder, 3)
```

```
## # A tibble: 3 × 8
##   country      continent year lifeExp      pop gdpPercap var1      var2
##   <fct>        <fct>    <int>   <dbl>    <int>    <dbl> <chr>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779. Valor fijo      7
## 2 Afghanistan Asia      1957   30.3  9240934    821. Valor fijo      7
## 3 Afghanistan Asia      1962   32.0 10267083    853. Valor fijo      7
```

```
## Podemos escribir lo mismo de distinta manera:
data_gapminder <- mutate(data_gapminder, var1 = "Valor fijo",
                          var2 = 7)
```

Recodificar variables con mutate()

Con `mutate()` también podemos realizar operaciones sobre variables ya existentes:

```
## Podemos recodificar usando variables y operadores aritméticos
# Calculemos el pbi total (pbi per capita * población)
d_gap <- mutate(gapminder, gdp = gdpPercap * pop)
head(d_gap, 3)
```

```
## # A tibble: 3 × 7
##   country    continent year lifeExp      pop gdpPercap      gdp
##   <fct>      <fct>   <int>  <dbl>   <int>   <dbl>   <dbl>
## 1 Afghanistan Asia     1952   28.8  8425333    779. 6567086330.
## 2 Afghanistan Asia     1957   30.3  9240934    821. 7585448670.
## 3 Afghanistan Asia     1962   32.0 10267083    853. 8758855797.
```

```
# Podemos calcular el logaritmo
d_gap <- mutate(d_gap, gdp_log = log(gdp))
head(d_gap, 2)
```

```
## # A tibble: 2 × 8
##   country    continent year lifeExp      pop gdpPercap      gdp gdp_log
##   <fct>      <fct>   <int>  <dbl>   <int>   <dbl>   <dbl> <dbl>
## 1 Afghanistan Asia     1952   28.8  8425333    779. 6567086330.    22.6
## 2 Afghanistan Asia     1957   30.3  9240934    821. 7585448670.    22.7
```

Transformaciones de tipo

Al igual que hacíamos con los vectores, podemos transformar el tipo de una variable

```
# Exploro tipo de variables
glimpse(d_gap)
```

```
## Rows: 1,704
## Columns: 3
## $ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ...
## $ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ...
## $ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8...
```

```
# Variable continente a caracteres y año a factor
d_gap <- d_gap %>%
  mutate(continent = as.character(continent),
         year = as.factor(year))

glimpse(d_gap)
```

```
## Rows: 1,704
## Columns: 3
## $ continent <chr> "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asi...
## $ year      <fct> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ...
## $ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8...
```

Recodificaciones condicionales

Recodificaciones condicionales

- Muchas veces transformar los datos implica recodificar una variable de forma condicional, esto es, asignar distintos valores en función de los valores de una o más variables.
- Para esto podemos utilizar las funciones; `ifelse()` (R Base), `mutate()`, `recode()` y `case_when()` (Tidyverse)
- Nosotros veremos recodificaciones condicionales con `case_when()` y `mutate()`

Recodificación condicional con `case_when` y `mutate`

Podemos crear variables condicionales con `case_when()` del paquete `dplyr`. Esencialmente, con `ifelse()` (R Base) podemos lograr lo mismo que con `case_when()` (Tidyverse). `case_when()` puede resultar más sencilla de utilizar al no haber necesidad de anidar la función cuando establecemos múltiples condiciones.

Cuando trabajamos con dataframes `case_when()` se utiliza dentro de `mutate()`. `case_when()` testea condiciones en orden (esto es importante cuando pasamos condiciones no excluyentes). `case_when()` lista condiciones para las que asigna un valor en caso de que sean verdaderas, y permite pasar múltiples condiciones. `TRUE` refiere a las condiciones no listadas. La estructura de `case_when()` es:

```
data %>%  
  mutate(var_nueva = case_when(var_original == "Valor 1" ~ "Valor A",  
                                var_original == "Valor 2" ~ "Valor B",  
                                TRUE ~ "Otros"))
```

Recodificación condicional con case_when y mutate

```
d_gap <- gapminder  
  
# Creemos una variable que indique si el país es Uruguay o no  
d_gap <- d_gap %>%  
  mutate(uruono = case_when(  
    country == "Uruguay" ~ "Si",  
    TRUE ~ "No")  
  )  
  
table(d_gap$uruono)
```

```
##  
##   No   Si  
## 1692   12
```

Recodificación condicional con case_when y mutate

Podemos establecer varias condiciones fácilmente:

```
d_gap <- gapminder
d_gap <- d_gap %>%
  mutate(mercosur = case_when(country == "Uruguay" ~ 1,
                              country == "Argentina" ~ 1,
                              country == "Paraguay" ~ 1,
                              country == "Brazil" ~ 1,
                              TRUE ~ 0))

table(d_gap$mercosur)
```

```
##
##      0      1
## 1656   48
```


Recodificación condicional con case_when y mutate

También podríamos usar operadores para simplificar esto:

```
d_gap <- d_gap %>%
  mutate(mercosur = case_when(
    country %in% c("Argentina", "Paraguay", "Brazil", "Uruguay") ~ 1,
    TRUE ~ 0)
)

d_gap <- d_gap %>%
  mutate(mercosur2 = case_when(
    country == "Argentina" | country == "Paraguay" |
    country == "Brazil" | country == "Uruguay" ~ 1,
    TRUE ~ 0)
)

identical(d_gap$mercosur, d_gap$mercosur2)
```

```
## [1] TRUE
```

Recodificación condicional con case_when y mutate

Recodificación con valores numéricos. Supongamos que queremos crear una nueva variable pob_rec, que clasifica a los países en población grande (más de 20 millones), mediana (entre 5 y 20) o pequeña (menos de 5)

```
d_gap <- d_gap %>%  
  mutate(pob_rec = case_when(  
    pop >= 20000000 ~ "Grande",  
    pop >= 5000000 & pop < 20000000 ~ "Mediana",  
    pop < 5000000 ~ "Pequeña",  
    TRUE ~ "Error")  
  )  
table(d_gap$pob_rec)
```

```
##  
## Grande Mediana Pequeña  
##      419      580      705
```

Recodificación condicional con case_when y mutate

`case_when()` sirve también para recodificar una variable con condiciones basadas en múltiples variables.

Supongamos que queremos una variable que indique los países-año con expectativa de vida mayor a 75 o pbi per cápita mayor a 20.000

```
d_gap <- d_gap %>%  
  mutate(var1 = case_when(gdpPercap > 20000 ~ 1,  
                           lifeExp > 75 ~ 1,  
                           TRUE ~ 0))  
table(d_gap$var1)
```

```
##  
##      0      1  
## 1493  211
```

Ejercicio

(1) Abran un script nuevo y tomen el dataframe de gapminder (importarlo de la carpeta data o descargarlo desde el paquete gapminder como debajo, y filtrar para la base para quedarnos solamente con las observaciones América y Oceanía

```
library(gapminder)  
gapminder <- gapminder
```

(2) En ese mismo dataframe con solamente las observaciones de América y Oceanía seleccionar solamente dos variables: country y lifeExp

(3) En ese mismo dataframe, crear una nueva variable que divida la expectativa de vida en 3 categorías: "Alta" (mayor o igual a 75 años); "Media" (mayor o igual a 65 y menor a 75 y "Baja" (menor a 65 años)

(4) Asegurarse de hacer los primeros 3 pasos en un mismo pipeline

Estadística descriptiva

Medidas de tendencia central

```
mean(data_gapminder$lifeExp) # Media
```

```
## [1] 59.47444
```

```
median(data_gapminder$lifeExp) # Mediana
```

```
## [1] 60.7125
```

```
sd(data_gapminder$lifeExp) # Desvío estandar
```

```
## [1] 12.91711
```

```
range(data_gapminder$lifeExp) # Rango
```

```
## [1] 23.599 82.603
```

```
max(data_gapminder$lifeExp)
```

```
## [1] 82.603
```

```
min(data_gapminder$lifeExp)
```

```
## [1] 23.599
```

Histogramas

También podemos graficar los datos rápidamente. Por ejemplo, un histograma:

```
hist(data_gapminder$lifeExp,  
main = "Distribución de expectativa de vida (Gapminder)")
```

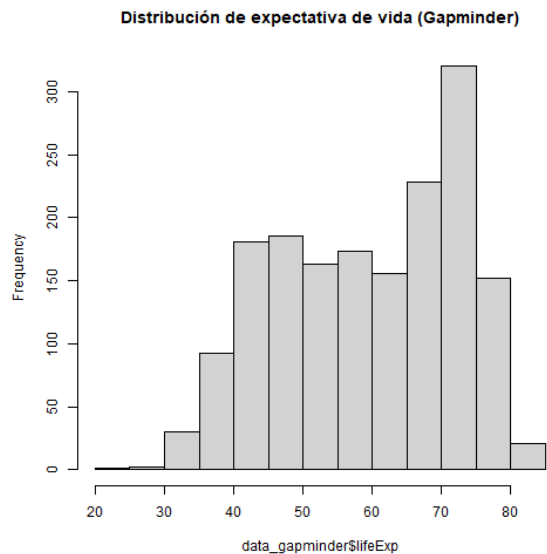
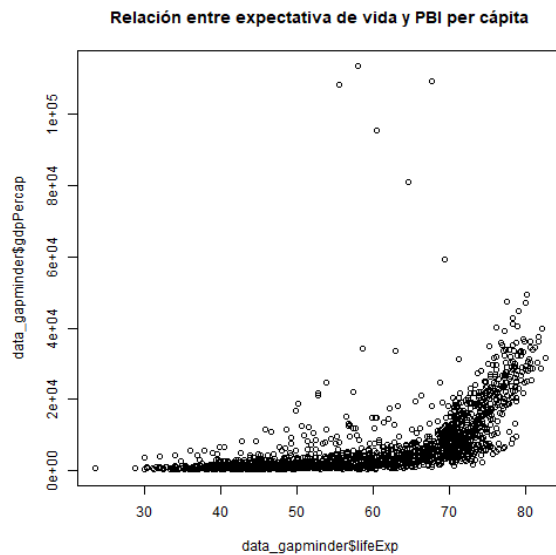


Gráfico de dispersión (scatterplot)

```
plot(data_gapminder$lifeExp, data_gapminder$gdpPercap,  
main = "Relación entre expectativa de vida y PBI per cápita")
```



Tablas simples

En R Base la función para obtener frecuencias es `table()` junto con `prop.table()` y `addmargins()`

`.codefontchico[`

```
# Para obtener una tabla de frecuencias de una variable usamos la función
# table() de R Base
tabla_1 <- table(data_gapminder$continent) # Frecuencia simple
tabla_1
```

```
##
##   Africa Americas      Asia  Europe Oceania
##     624      300      396     360      24
```

```
prop.table(tabla_1) # Proporciones
```

```
##
##   Africa  Americas      Asia  Europe  Oceania
## 0.36619718 0.17605634 0.23239437 0.21126761 0.01408451
```

```
addmargins(tabla_1) # Totales
```

```
##
##   Africa Americas      Asia  Europe Oceania      Sum
##     624      300      396     360      24     1704
```

```
addmargins(prop.table(tabla_1)) # Proporciones y totales
```

```
##
##   Africa  Americas      Asia  Europe  Oceania      Sum
## 0.36619718 0.17605634 0.23239437 0.21126761 0.01408451 1.00000000
```

Tablas cruzadas

Resumir datos: tablas simples

Con `count()` podemos hacer lo mismo que con `table()`, pero en el contexto del tidyverse. Con un simple `mutate()` podemos transformar la frecuencia simple en proporción o porcentaje

```
# Frecuencia simple
d_gap %>%
  count(continent)
```

```
## # A tibble: 5 × 2
##   continent     n
##   <fct>      <int>
## 1 Africa       624
## 2 Americas     300
## 3 Asia         396
## 4 Europe       360
## 5 Oceania       24
```

```
# Frecuencia proporción
d_gap %>%
  count(continent) %>%
  mutate(per = n/sum(n)*100) %>% # Porcentaje
  mutate(prop = n/sum(n)) # Proporción
```

```
## # A tibble: 5 × 4
##   continent     n   per   prop
##   <fct>      <int> <dbl> <dbl>
## 1 Africa       624  36.6  0.366
## 2 Americas     300  17.6  0.176
## 3 Asia         396  23.2  0.232
## 4 Europe       360  21.1  0.211
## 5 Oceania       24   1.41  0.0141
```

Resumir datos: tablas cruzadas

Cuando usamos `count()` con dos variables vemos que la salida está en formato largo. Eso nos puede servir para graficar si usamos `ggplot2` y para otros paquetes, pero no es como generalmente queremos leer una tabla para interpretarla.

```
# Tabla cruzada en formato largo
d_gap %>%
  count(continent, pob_rec)
```

```
## # A tibble: 15 × 3
##   continent pob_rec     n
##   <fct>      <chr> <int>
## 1 Africa    Grande     86
## 2 Africa    Mediana    208
## 3 Africa    Pequeña    330
## 4 Americas Grande     72
## 5 Americas Mediana     98
## 6 Americas Pequeña    130
## 7 Asia      Grande    168
## 8 Asia      Mediana    111
## 9 Asia      Pequeña    117
## 10 Europe   Grande     92
## 11 Europe   Mediana    152
## 12 Europe   Pequeña    116
## 13 Oceania  Grande      1
## 14 Oceania  Mediana     11
## 15 Oceania  Pequeña     12
```

Resumir datos: tablas cruzadas

Para pasar a formato ancho podemos utilizar la función `spread()`. El primer argumento dentro de `spread()` es la variable que queremos que pase a ser columnas y el segundo argumento la variable que contiene los valores.

```
# Tabla cruzada en formato ancho
d_gap %>%
  count(continent, pob_rec) %>%
  spread(pob_rec, n)
```

```
## # A tibble: 5 × 4
##   continent Grande Mediana Pequeña
##   <fct>      <int>   <int>   <int>
## 1 Africa         86     208     330
## 2 Americas        72      98     130
## 3 Asia         168     111     117
## 4 Europe          92     152     116
## 5 Oceania         1      11      12
```

Resumir datos: tablas cruzadas

Cuando analizamos tablas cruzadas, muchas veces queremos ver porcentajes en lugar de frecuencias. Para ello podemos transformar la frecuencia en porcentaje.

```
# Porcentaje total
d_gap %>%
  count(continent, pob_rec) %>%
  mutate(n = n/sum(n)*100) %>%
  spread(pob_rec, n)
```

```
## # A tibble: 5 × 4
##   continent Grande Mediana Pequeña
##   <fct>      <dbl>   <dbl>   <dbl>
## 1 Africa     5.05    12.2    19.4
## 2 Americas  4.23     5.75     7.63
## 3 Asia      9.86     6.51     6.87
## 4 Europe    5.40     8.92     6.81
## 5 Oceania   0.0587   0.646   0.704
```

Resumir datos: tablas cruzada

Cuando trabajamos con porcentajes en tablas cruzadas, tenemos que considerar el total del %. En el ejemplo anterior el porcentaje era sobre el total, muchas veces queremos calcularlo sobre cada fila o cada columna

```
# % de países en cada tamaño de población por continente
d_gap %>%
  count(continent, pob_rec) %>%
  mutate(n = n/sum(n)*100, .by = continent) %>%
  spread(pob_rec, n)
```

```
## # A tibble: 5 × 4
##   continent Grande Mediana Pequeña
##   <fct>      <dbl>   <dbl>   <dbl>
## 1 Africa      13.8     33.3     52.9
## 2 Americas    24       32.7     43.3
## 3 Asia        42.4     28.0     29.5
## 4 Europe      25.6     42.2     32.2
## 5 Oceania      4.17     45.8     50
```

```
# % de continente por tamaño de población
d_gap %>%
  count(continent, pob_rec) %>%
  mutate(n = n/sum(n)*100, .by = pob_rec) %>%
  spread(pob_rec, n)
```

```
## # A tibble: 5 × 4
##   continent Grande Mediana Pequeña
##   <fct>      <dbl>   <dbl>   <dbl>
## 1 Africa      20.5     35.9     46.8
## 2 Americas    17.2     16.9     18.4
## 3 Asia        40.1     19.1     16.6
## 4 Europe      22.0     26.2     16.5
## 5 Oceania      0.239     1.90     1.70
```


Ejercicio

- (1) Importar el archivo excel en la carpeta data llamado "datauru"*
- (2) Explorar las variables y el tipo de cada variable con la función glimpse()*
- (3) Calcular la media de una variable numérica*
- (4) Crear una tabla de proporciones con la distribución de la variable partido*
- (5) Crear un gráfico de dispersión con las variables inflación (eje de las x) y aprobación (eje de las y)*

Tablas con estadísticas descriptivas

Resumir datos

Resumir datos con estadísticos descriptivos es una de las partes fundamentales del análisis de datos. Para ello utilizaremos la función `summarise()` o `summarize()`, muchas veces en conjunto con `group_by()`.

Escencialmente `summarise()` resume un dataframe en una fila según una estadística especificada. Por ejemplo, calculando la media de una variable

```
gapminder %>%  
  summarise(media = mean(lifeExp, na.rm=T))
```

```
## # A tibble: 1 × 1  
##   media  
##   <dbl>  
## 1  59.5
```

```
# Por ahora no hay mucha diferencia con  
mean(gapminder$lifeExp, na.rm = TRUE)
```

```
## [1] 59.47444
```

Resumir datos

Hasta ahora `summarise()` no nos es de gran utilidad, la utilidad de `summarise()` es su uso conjunto con `group_by()`, para estimar diferentes estadísticas según grupos específicos.

Cuando utilizamos `group_by()` en un pipeline cambiamos la unidad de análisis desde todo el dataframe a niveles de una variable. Retomando el ejemplo, podemos ver el promedio de expectativa de vida según año:

```
gapminder %>%  
  group_by(year) %>%  
  summarise(media = mean(lifeExp, na.rm = T))
```

```
## # A tibble: 12 × 2  
##   year media  
##   <int> <dbl>  
## 1  1952  49.1  
## 2  1957  51.5  
## 3  1962  53.6  
## 4  1967  55.7  
## 5  1972  57.6  
## 6  1977  59.6  
## 7  1982  61.5  
## 8  1987  63.2  
## 9  1992  64.2  
## 10 1997  65.0  
## 11 2002  65.7  
## 12 2007  67.0
```

Resumir datos

Algunas de las operaciones más utilizadas para resumir datos:

- `mean()`: media
- `median()`: mediana
- `sd()`: desvío estandar
- `sum()`: suma
- `n()`: número de observaciones
- `n_distinct()`: número de valores únicos
- `min()` y `max()`: mínimo y máximo
- `first()`: primer valor

Resumir datos

Podemos utilizar más de una variable dentro de `group_by()`. Por ejemplo, calculemos la media de expectativa de vida por año comparando America y Europa para 1997, 2002 y 2007:

```
resumen_1 <- gapminder %>%  
  filter(continent %in% c("Americas", "Europe")) %>%  
  filter(year >= 1997) %>%  
  group_by(continent, year) %>%  
  summarise(media = mean(lifeExp, na.rm = TRUE))
```

```
## `summarise()` has grouped output by 'continent'. You can override using the  
## `.groups` argument.
```

```
resumen_1
```

```
## # A tibble: 6 × 3  
## # Groups:   continent [2]  
##   continent year media  
##   <fct>      <int> <dbl>  
## 1 Americas  1997  71.2  
## 2 Americas  2002  72.4  
## 3 Americas  2007  73.6  
## 4 Europe    1997  75.5  
## 5 Europe    2002  76.7  
## 6 Europe    2007  77.6
```

Resumir datos

Una de las grandes ventajas de `summarise()` es que podemos resumir muy fácilmente varias estadísticas en un solo dataframe.

```
resumen_2 <- gapminder %>%  
  filter(continent %in% c("Americas", "Europe")) %>%  
  filter(year == 2007) %>%  
  group_by(continent) %>%  
  summarise(media = mean(gdpPercap),  
            desvio = sd(gdpPercap),  
            suma = sum(gdpPercap),  
            max = max(gdpPercap),  
            min = min(gdpPercap),  
            paises = n())  
  
resumen_2
```

```
## # A tibble: 2 × 7  
##   continent media desvio   suma   max   min paises  
##   <fct>      <dbl> <dbl>   <dbl> <dbl> <dbl> <int>  
## 1 Americas  11003.  9713. 275076. 42952. 1202.    25  
## 2 Europe    25054. 11800. 751634. 49357.  5937.    30
```

Exportar tablas

Las tablas y resúmenes de salida de tidyverse son dataframes. Como tales, los podemos exportar fácilmente a otros tipos de archivos. Veamos como exportar una tabla con la función `write_xlsx()` del paquete `writexl`. Ya habíamos creado la tabla "resumen_2", entonces con una sola línea de código la podemos guardar. La función `write_xlsx()` tiene dos argumentos principales, el dataframe a ser guardado y el directorio donde ser guardado junto con el nombre del archivo que vamos a crear. Tener en cuenta que si el directorio y nombre de un archivo coincide con el de otro archivo ya existente en la computadora, lo sobreescribirá.

```
library(writexl)
writexl::write_xlsx(resumen_2, "resultados/mi_primera_tabla.xlsx")
```


Ejercicio

Para este ejercicio vamos a utilizar el dataframe de gapminder y vamos a crear una variable nueva recodificando los valores la variable de pbi per capita (gdpPercap). Para eso peguen el código debajo

```
library(gapminder)
gapminder <- gapminder %>%
  mutate(gdp_rec = case_when(
    gdpPercap < 2000 ~ "Bajo",
    gdpPercap >= 2000 & gdpPercap <= 5000 ~ "Medio",
    gdpPercap > 5000 ~ "Alto"
  ))
```

- (1) Utilizar count() para crear una tabla con el % de casos en cada categoría de la recodificación de pbi per cápita*
- (2) Ahora crear una tabla cruzada entre continente y la variable recodificada de pbi per cápita (incluir el % a nivel de fila)*
- (3) Crear una tabla resumen con la media de la expectativa de vida y la cantidad de casos según continente*